

Shape-safe vector autograd

Backprop is a right-to-left chain of vector-Jacobian products.

THE QUESTION

How do we backpropagate through vectors and matrices without building giant Jacobians or getting the shapes wrong?

THE INTUITION

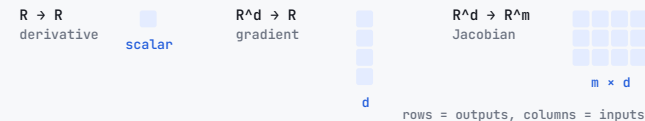
Each layer receives the upstream gradient, one number per output, and sends it back through the transpose of its local Jacobian. For a dense layer that transpose is just $W^\top$, so the full Jacobian is never built.

KEY TAKEAWAY

Every tensor's gradient has the same shape as the tensor. Predict the shape, then compute: $g_x = W^\top g_z, g_W = g_z x^\top, g_b = g_z$.

$x \rightarrow z = Wx + b \rightarrow \text{loss} \rightarrow \text{upstream } g_z \rightarrow \text{VJP: } W^\top g_z \rightarrow g_W = g_z x^\top, g_b = g_z$

1 Name the local derivative by shape



map
 $u \in \mathbb{R} \rightarrow v \in \mathbb{R}$
 $x \in \mathbb{R}^d \rightarrow z \in \mathbb{R}$
 $x \in \mathbb{R}^d \rightarrow z \in \mathbb{R}^m$

local object
 $\partial v / \partial u$
 $\nabla_x z$
 $J_{ij} = \partial z_i / \partial x_j$

shape
scalar
 d
 $m \times d$

Rows of J correspond to outputs; columns to inputs.

2 The vector chain rule

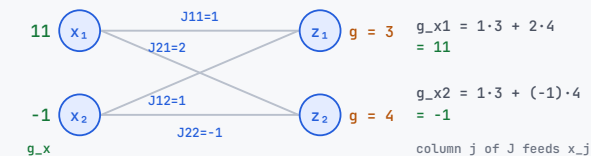


For $x \in \mathbb{R}^d \rightarrow z \in \mathbb{R}^m \rightarrow \mathcal{L} \in \mathbb{R}$:

$$\nabla_x \mathcal{L} = \underbrace{J^\top}_{d \times m} \underbrace{\nabla_z \mathcal{L}}_{m \times 1}$$

The arriving $g_z = \nabla_z \mathcal{L}$ is the upstream gradient;
 $J^\top g_z$ is a **vector-Jacobian product (VJP)**.
Frameworks compute VJPs directly and usually never build the full Jacobian.

3 Why the transpose appears



If $z = (x_1 + x_2, 2x_1 - x_2)$ then $J = \begin{pmatrix} 1 & 1 \\ 2 & -1 \end{pmatrix}$. With upstream $g_z = (3, 4)$:

$$g_x = J^\top g_z = \begin{pmatrix} 1 & 2 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 11 \\ -1 \end{pmatrix}$$

Each input adds the contribution arriving through every output path.

4 Seed rule

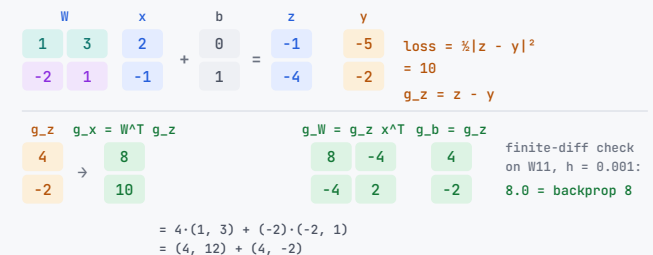
A scalar loss starts reverse mode with $\partial \mathcal{L} / \partial \mathcal{L} = 1$. A non-scalar output needs an explicit seed vector v ; backward then returns $J^\top v$.

5 Scalar affine node

$z = w^\top x + b$ has local gradients $(\partial z) / (\partial w) = x$, $(\partial z) / (\partial x) = w$, $(\partial z) / (\partial b) = 1$. Multiply each by the arriving g_z :

$$g_w = g_z x, \quad g_x = g_z w, \quad g_b = g_z$$

6 Dense layer, one example



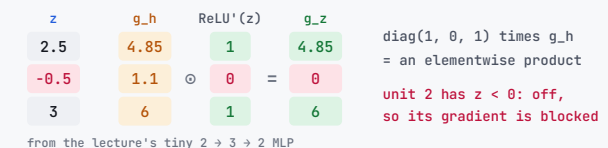
teal row = neuron 1 · purple row = neuron 2 · shapes: g_W is 2×2 like W

Forward $z = Wx + b$ with $W \in \mathbb{R}^{m \times d}$, $x \in \mathbb{R}^d$, $z, b \in \mathbb{R}^m$. Backward for upstream $g_z \in \mathbb{R}^m$:

$$g_x = W^\top g_z, \quad g_b = g_z, \quad g_W = g_z x^\top$$

The outer product gives the parameter-gradient matrix: $(m \times 1)(1 \times d) = m \times d$. Above, $\mathcal{L} = \frac{1}{2} \|z - y\|^2$, so $g_z = z - y$.

7 Elementwise nonlinearities



from the lecture's tiny $2 \rightarrow 3 \rightarrow 2$ MLP

If $h = \varphi(z)$ elementwise, the local Jacobian is diagonal, so the VJP is an elementwise product: $g_z = g_h \odot \varphi'(z)$, with $\text{ReLU}'(z) = \mathbb{1}[z > 0]$ (for $z \neq 0$) and $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Matrices, batches, and autograd traps

Use shape contracts to make vectorized backward passes auditable.

8 Dense layer, row-batch convention

$$\begin{array}{|c|c|} \hline X & Z \\ \hline 2 & -1 \\ -1 & 2 \\ \hline \end{array} \rightarrow \begin{array}{|c|c|} \hline Z & Y \\ \hline -1 & -4 \\ 5 & 5 \\ \hline \end{array} \quad \begin{array}{|c|c|} \hline Y & G_Z \\ \hline -5 & -2 \\ 7 & 1 \\ \hline \end{array} \quad G_Z = (Z - Y) / B$$

per-example losses 10, 10 → mean 10; the 1/B lives in G_Z

$$\begin{array}{|c|c|} \hline g_W & g_b \\ \hline 5 & -4 \\ -4 & 5 \\ \hline \end{array} \quad \begin{array}{|c|} \hline g_b \\ \hline 1 \\ \hline \end{array} \quad \begin{array}{|c|c|} \hline g_X \\ \hline 4 & 5 \\ -5 & -1 \\ \hline \end{array}$$

g_W = mean of per-example outer products $r_n x_n^T$: $((8, -4; -4, 2) + [2, -4; -4, 8]) / 2$
 g_b = column sums of G_Z

Store examples as rows of $X \in \mathbb{R}^{N \times d}$: $Z = XW^T + 1b^T \in \mathbb{R}^{N \times m}$. For upstream $G_Z \in \mathbb{R}^{N \times m}$:

$$G_X = G_Z W, \quad G_W = G_Z^T X, \quad g_b = \sum_{n=1}^N G_{Z,n}$$

If the loss is a mean over examples, its upstream signal already carries the factor $1/N$.

9 Broadcasting means reduction in backward

$$\begin{array}{|c|c|} \hline \text{forward: copy } b \text{ to every row} \\ \hline 0 & 1 \\ \hline \end{array} \rightarrow \begin{array}{|c|c|} \hline 0 & 1 \\ 0 & 1 \\ \hline \end{array} \quad \begin{array}{|c|c|} \hline \text{backward: sum over the copied axis} \\ \hline 2 & -1 \\ -1 & 2 \\ \hline \end{array} \rightarrow \begin{array}{|c|c|} \hline 1 & 1 \\ \hline \end{array}$$

trap: y of shape $[N]$ minus $\hat{y}$ of shape $[N, 1]$ broadcasts to $[N, N]$
 a plausible-looking but wrong loss: assert shapes first

Forward copies b across N rows; backward undoes the copy by summing: $g_b = G_Z^T 1$. General rule: if forward broadcasting inserted or expanded an axis, sum the gradient over that axis in backward.

10 Branches accumulate

If x feeds two paths a and c , both contribute:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial x} + \frac{\partial \mathcal{L}}{\partial c} \frac{\partial c}{\partial x}$$

Reverse mode adds all arriving gradients at the shared node. Reusing a tensor is not “last path wins”.

11 Matrix multiplication mnemonic

$$\begin{array}{|c|} \hline G_C \\ \hline 3 \times 4 \\ \hline \end{array} \cdot \begin{array}{|c|} \hline B^T \\ \hline 4 \times 2 \\ \hline \end{array} = \begin{array}{|c|} \hline G_A \\ \hline 3 \times 2 \\ \hline \end{array} \quad \begin{array}{|c|} \hline A^T \\ \hline 2 \times 3 \\ \hline \end{array} \cdot \begin{array}{|c|} \hline G_C \\ \hline 3 \times 4 \\ \hline \end{array} = \begin{array}{|c|} \hline G_B \\ \hline 2 \times 4 \\ \hline \end{array}$$

A is 3×2 , B is 2×4 (illustrative): G_A matches A , G_B matches B

For $C = AB$ and upstream G_C : $G_A = G_C B^T$, $G_B = A^T G_C$. Keep the untouched neighbour and transpose it so the dimensions fit; then check G_A matches A and G_B matches B .

12 PyTorch reverse-mode checklist

- ✓ Leaf tensors that need gradients use `requires_grad=True`.
- ✓ The forward graph is built by tracked tensor operations.
- ✓ Call `loss.backward()` on a scalar loss.
- ✓ For vector outputs, supply the VJP seed explicitly.
- ✓ Read leaf gradients from `.grad`; clear them before reuse.
- ✓ Update parameters under `no_grad()` or through an optimizer.

13 Minimal VJP check

```
x = torch.tensor([1., 2.])
x.requires_grad_()
y = f(x) # shape [m]
v = torch.randn_like(y)
y.backward(v) # x.grad = J.T @ v
```

For tiny functions, compare against a full Jacobian or finite differences. Agreement must hold in both value and shape.

14 Failure modes

symptom	likely cause
wrong orientation	mixed row/column conventions; missing transpose
gradient too large	forgot the mean factor, or summed a branch twice
bias gradient wrong	forgot the reduction over broadcast axes
<code>.grad</code> doubles	did not clear accumulated gradients
None gradient	detached graph, non-leaf tensor, or unused path
shapes fine, loss wrong	unintended broadcasting

15 Final shape audit

Forward: write the shape after every operation.
Backward: start from the loss and move right to left.
At each node: multiply by the local derivative, then sum contributions.
Invariant: a returned gradient has its input's shape.

16 Put it together: the lecture's tiny $2 \rightarrow 3 \rightarrow 2$ MLP

Forward with $x = (2, -1)$, $y = (1, 0)$: $z_1 = W_1 x + b_1 = (2.5, -0.5, 3)$, $h = \text{ReLU}(z_1) = (2.5, 0, 3)$, $\hat{y} = W_2 h + b_2 = (4, 1.75)$, $\mathcal{L} = \|\hat{y} - y\|^2 = 12.0625$ (sum of squares, not halved).

Backward: $g_{\hat{y}} = 2(\hat{y} - y) = (6, 3.5)$, $g_h = W_2^T g_{\hat{y}} = (4.85, 1.1, 6)$, $g_{z_1} = g_h \odot (1, 0, 1) = (4.85, 0, 6)$, $g_x = W_1^T g_{z_1} = (10.85, -8.425)$. $G_{W_1} = g_{z_1} x^T$ has a zero row: the off unit learns nothing this step.

Backprop is scalar chain-rule reasoning, stacked and vectorized without changing the logic

Numbers from the Lecture 8 dense-layer example, batch and tiny MLP, each checked against finite differences. Slides and summary video: nipunbatra.github.io/dl-2026.